

🎯 ASEEL — Multimodal Deep Learning for Counterfeit Luxury Bag Detection

Architecture:

- **Image Branch:** EfficientNet-B0 (pretrained) → 1,280 features
- **Tabular Branch:** Label Encoding + Standard Scaling → 7 features
- **Text Branch:** BERT (bert-base-uncased) → 768 features
- **OCR Branch:** EasyOCR → BERT → 768 features
- **Fusion:** Concatenation → MLP (512 → 128 → 1) → Sigmoid

Dataset:

- 1,087 products | 6 brands | 522 fake + 565 real
- Images: No-background (rembg processed)
- Evaluation: 5-Fold CV + Test Set + External Test (mahally.com)

```
# =====
# 📦 CELL 1: Install Dependencies
# =====
!pip install -q torch torchvision transformers easyocr scikit-learn

import warnings
warnings.filterwarnings('ignore')
print("✅ All packages installed!")
```

```
2.9/2.9 MB 31.1 MB/s eta 0s
180.7/180.7 kB 12.9 MB/s eta 0s
978.2/978.2 kB 60.2 MB/s eta 0s
299.6/299.6 kB 24.1 MB/s eta 0s
✅ All packages installed!
```

```
# =====
# 📖 CELL 2: Import Libraries
# =====
import os
import gc
import glob
```

```
import re
import numpy as np
import pandas as pd
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from tqdm import tqdm
from collections import defaultdict

# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torch.nn.functional as F

# Torchvision
import torchvision.transforms as transforms
from torchvision import models

# Transformers (BERT)
from transformers import BertTokenizer, BertModel

# OCR
import easyocr

# Scikit-learn
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score,
    accuracy_score, f1_score, roc_curve, precision_recall_curve
)

# Set device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memory: {torch.cuda.get_device_properties(0).total_memory}")

# Set seeds for reproducibility
SEED = 42
torch.manual_seed(SEED)
```

```

np.random.seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

print("✅ All libraries imported!")

```

 Device: cuda
 GPU: Tesla T4
 Memory: 15.6 GB
 ✅ All libraries imported!

```

# =====
# 📁 CELL 3: Mount Google Drive & Setup Paths
# =====

from google.colab import drive
drive.mount('/content/drive')

# — Base Paths (عدّل لو مختلف عندك) —
BASE_PATH = '/content/drive/MyDrive'
DATA_PATH = BASE_PATH
IMG_BASE = os.path.join(BASE_PATH, 'product_images')

# — Image Folder Mapping (nobg folders only) —
IMAGE_FOLDERS = {
    'Chanel': {
        'real': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
        'fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
    },
    'Coach': {
        'real': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
        'fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
    },
    'Louis Vuitton': {
        'real': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
        'fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
    },
    'YSL': {
        'real': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
        'fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
    },
    'Fendi': {
        'real': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
        'fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/product',
    },
    'Prada': {

```

```

        'real': '/content/drive/MyDrive/GP2 (أصيل)/Data/produc
        'fake': '/content/drive/MyDrive/GP2 (أصيل)/Data/produc
    },
}

# — Verify Paths —
print("📁 Verifying image folders...")
total_all = 0
for brand, paths in IMAGE_FOLDERS.items():
    for label, path in paths.items():
        if os.path.exists(path):
            n_imgs = len([f for f in os.listdir(path) if f.lower
            print(f" ✅ {brand} {label}: {n_imgs} images")
            total_all += n_imgs
        else:
            print(f" ❌ {brand} {label}: NOT FOUND → {path}")

print(f"\n 📊 Total images found: {total_all}")
print("✅ Paths configured!")

```

```

Mounted at /content/drive
📁 Verifying image folders...
✅ Chanel real: 517 images
✅ Chanel fake: 363 images
✅ Coach real: 560 images
✅ Coach fake: 388 images
✅ Louis Vuitton real: 396 images
✅ Louis Vuitton fake: 54 images
✅ YSL real: 410 images
✅ YSL fake: 592 images
✅ Fendi real: 417 images
✅ Fendi fake: 305 images
✅ Prada real: 509 images
✅ Prada fake: 527 images

📊 Total images found: 5038
✅ Paths configured!

```

```
# =====
# 📊 CELL 4: Load Data
# =====

products = pd.read_excel(os.path.join(DATA_PATH, '/content/drive/
reviews = pd.read_excel(os.path.join(DATA_PATH, '/content/drive

print(f"📦 Products: {len(products)} ({(products['is_authentic
print(f"💬 Reviews: {len(reviews)}")
print(f"\n📊 Brand Distribution:")
print(pd.crosstab(products['brand'], products['is_authentic'],

# — Aggregate Reviews per Product —
review_agg = reviews.groupby('Id')['Reviews_text'].apply(
    lambda x: ' '.join([str(r) for r in x if str(r) != 'no_rev
).reset_index()
review_agg.columns = ['Id', 'combined_reviews']
review_agg['combined_reviews'] = review_agg['combined_reviews']

# Merge
data = products.merge(review_agg, on='Id', how='left')
data['combined_reviews'] = data['combined_reviews'].fillna('no_

print(f"\n✅ Data merged: {len(data)} products")
print(f"    With reviews: {(data['combined_reviews'] != 'no_rev
print(f"    Without reviews: {(data['combined_reviews'] == 'no_
```

📦 Products: 1087 (522 fake, 565 real)
 💬 Reviews: 3359

📊 Brand Distribution:

is_authentic	0	1	All
brand			
Chanel	94	94	188
Coach	95	94	189
Fendi	95	93	188
Louis Vuitton	48	95	143
Prada	94	94	188
YSL	96	95	191
All	522	565	1087

✅ Data merged: 1087 products
 With reviews: 420
 Without reviews: 667

```
# =====
```

```

#  CELL 5: Build Image Mapping (Product ID → Image Paths)
# =====

def extract_id_from_filename(filename):
    """Extract product ID from image filename."""
    numbers = re.findall(r'(\d+)', filename)
    if numbers:
        for num in numbers:
            num_int = int(num)
            if num_int >= 200:
                return num_int
        return int(numbers[0])
    return None

# Build mapping: product_id → [list of image paths]
product_images = defaultdict(list)
valid_extensions = {'.jpg', '.jpeg', '.png', '.webp'}

print("🔍 Scanning image folders...")
total_images = 0

for brand, paths in IMAGE_FOLDERS.items():
    for label, folder_path in paths.items():
        if not os.path.exists(folder_path):
            print(f"⚠️ Skipping {brand} {label}")
            continue

        files = os.listdir(folder_path)
        brand_count = 0

        for fname in files:
            ext = os.path.splitext(fname)[1].lower()
            if ext not in valid_extensions:
                continue

            product_id = extract_id_from_filename(fname)
            if product_id is not None and product_id in data['.':
                full_path = os.path.join(folder_path, fname)
                product_images[product_id].append(full_path)
                brand_count += 1
                total_images += 1

        print(f" {brand} {label}: {brand_count} images")

# Statistics

```

```

products_with_images = len(product_images)
imgs_per_product = [len(v) for v in product_images.values()]

print(f"\n📊 Summary:")
print(f"    Total images: {total_images}")
print(f"    Products with images: {products_with_images}/{len(data)}")
print(f"    Imgs/product: min={min(imgs_per_product)}, max={max(imgs_per_product)}")

# Filter to products with images only
data = data[data['Id'].isin(product_images.keys())].reset_index()
print(f"\n✅ Working dataset: {len(data)} products")

```

```

🔍 Scanning image folders...
Chanel real: 517 images
Chanel fake: 363 images
Coach real: 560 images
Coach fake: 388 images
Louis Vuitton real: 396 images
Louis Vuitton fake: 54 images
YSL real: 410 images
YSL fake: 592 images
Fendi real: 417 images
Fendi fake: 305 images
Prada real: 509 images
Prada fake: 527 images

📊 Summary:
Total images: 5038
Products with images: 1075/1087
Imgs/product: min=1, max=29, avg=4.7

✅ Working dataset: 1075 products

```

```

# =====
# 📄 CELL 6: OCR Feature Extraction (EasyOCR)
# =====

print("📄 Initializing EasyOCR...")
reader = easyocr.Reader(['en'], gpu=torch.cuda.is_available())

def extract_ocr_text(image_paths, max_images=3):
    """Extract text from product images using OCR."""
    all_text = []
    for img_path in image_paths[:max_images]:
        try:
            results = reader.readtext(img_path, detail=0)
            all_text.extend(results)

```

```
WARNING:easyocr.easyocr:Downloading detection model, please wait. Th
 Initializing EasyOCR...
Progress: |██████████████████████████████████████████████████| 100.0%
Progress: |██████████████████████████████████████████████████| 100.0%
OCR: 100%|██████████████████████████████████████████████████| 1075/1075 [44:34<00:00, 2.49s/it]
 OCR Complete!
    With text: 715/1075
    Without text: 360/1075

 Sample OCR (Fake):
1. no_text...
2. no_text...
3. no_text...

 Sample OCR (Real):
1. 3 7CI 38...
2. CHANEL MADE JN FRANCE ApFnc...
3. no text...
```



```
# =====
# 🔒 CELL 6b: Hold-Out Test Split (20% - أبداً ما يلمسها لنموذج ما يلمسها أبداً)
# =====
# نقسم البيانات إلى:
# • 80% → Cross-Validation (train + validation)
# • 20% → Test Set (يُختبر مرة واحدة فقط في النهاية)
# fake/real للحفاظ على نسبة stratified التقسيم

from sklearn.model_selection import train_test_split

cv_data, test_data_holdout = train_test_split(
    data,
    test_size=0.20,
    stratify=data['is_authentic'],
    random_state=SEED
)
cv_data = cv_data.reset_index(drop=True)
test_data_holdout = test_data_holdout.reset_index(drop=True)

print("=" * 70)
print("🔒 HOLD-OUT TEST SPLIT")
print("=" * 70)
print(f"📊 CV set (train+val): {len(cv_data)} products")
print(f"    • Fake: {(cv_data['is_authentic']==0).sum()} | Real: {(cv_data['is_authentic']==1).sum()}")
print(f"🔒 Test set (held-out): {len(test_data_holdout)} products")
print(f"    • Fake: {(test_data_holdout['is_authentic']==0).sum()} | Real: {(test_data_holdout['is_authentic']==1).sum()}")
print(f"\n✅ Test set is COMPLETELY separate – the model will NEVER see it during training")
print(f"✅ It will be evaluated ONCE at the end (Cell 13b) for honest test results")
```

```
=====
🔒 HOLD-OUT TEST SPLIT
=====
```

```
📊 CV set (train+val): 860 products
    • Fake: 409 | Real: 451
🔒 Test set (held-out): 215 products
    • Fake: 102 | Real: 113
```

```
✅ Test set is COMPLETELY separate – the model will NEVER see it during training
✅ It will be evaluated ONCE at the end (Cell 13b) for honest test results
```

```
# =====
# ⚙️ CELL 7: Feature Setup (NO FITTING – prevents data leakage)
# =====

# 🔧 FIX: هنا على كل البيانات fit_transform لا نعمل (يسبب data
# بدلاً من ذلك، فقط نحدد أسماء الأعمدة. الترميز والتجميع يتم
# فقط training fold على fit) CV loop داخل).

categorical_cols = ['Material', 'Closure_Type', 'Style', 'Pattern']

# أسماء الميزات الجدولية النهائية (6 ميزات)
tabular_features = ['price_scaled', 'Material_encoded', 'Closure_Type_encoded',
                    'Style_encoded', 'Pattern_encoded', 'brand_encoded']

print(f"📊 Tabular Features ({len(tabular_features)}):")
for f in tabular_features:
    print(f"    • {f}")

print(f"\n📋 Categorical columns to encode (per-fold): {categorical_cols}")
print(f"💰 Price range (raw): [{data['price'].min():.2f}, {data['price'].max():.2f}]")

print("\n🛡️ DATA LEAKAGE PREVENTION:")
print("    ✓ LabelEncoders will fit on TRAIN fold only")
print("    ✓ StandardScaler will fit on TRAIN fold only")
print("    ✓ Applied inside the CV loop (Cell 12)")
print("\n✅ Feature setup complete (no fitting yet)!")
```

📊 Tabular Features (6):

- price_scaled
- Material_encoded
- Closure_Type_encoded
- Style_encoded
- Pattern_encoded
- brand_encoded

📋 Categorical columns to encode (per-fold): ['Material', 'Closure_Type', 'Style', 'Pattern', 'brand']
 💰 Price range (raw): [4.99, 63685.00]

🛡️ DATA LEAKAGE PREVENTION:

- ✓ LabelEncoders will fit on TRAIN fold only
- ✓ StandardScaler will fit on TRAIN fold only
- ✓ Applied inside the CV loop (Cell 12)

✅ Feature setup complete (no fitting yet)!

```
# =====
```

```
# 📦 CELL 8: Custom Dataset Class
# =====

class LuxuryBagDataset(Dataset):
    def __init__(self, dataframe, product_images_dict, tokenizer,
                 tabular_features, img_transform=None, max_images=10):
        self.data = dataframe.reset_index(drop=True)
        self.product_images = product_images_dict
        self.tokenizer = tokenizer
        self.tabular_features = tabular_features
        self.img_transform = img_transform
        self.max_images = max_images
        self.max_len = max_len

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        row = self.data.iloc[idx]
        product_id = row['Id']

        # 1. IMAGES
        img_paths = self.product_images.get(product_id, [])
        images = []
        for path in img_paths[:self.max_images]:
            try:
                img = Image.open(path).convert('RGB')
                if self.img_transform:
                    img = self.img_transform(img)
                images.append(img)
            except:
                continue

        if len(images) == 0:
            images = [torch.zeros(3, 224, 224)]

        img_tensor = torch.stack(images[:self.max_images])
        if img_tensor.size(0) < self.max_images:
            padding = torch.zeros(self.max_images - img_tensor.size(0),
                                  3, 224, 224)
            img_tensor = torch.cat([img_tensor, padding], dim=0)

        num_valid_images = min(len(images), self.max_images)

        # 2. TABULAR
        tab = torch.tensor(row[self.tabular_features].values.astype(float))
```

```

# 3. REVIEW TEXT
review_encoding = self.tokenizer(
    str(row['combined_reviews']), max_length=self.max_
    padding='max_length', truncation=True, return_tenso
)

# 4. OCR TEXT
ocr_encoding = self.tokenizer(
    str(row['ocr_text']), max_length=128,
    padding='max_length', truncation=True, return_tenso
)

# 5. LABEL
label = torch.tensor(row['is_authentic'], dtype=torch.

return {
    'images': img_tensor,
    'num_valid_images': num_valid_images,
    'tabular': tab,
    'review_input_ids': review_encoding['input_ids'].squeeze(
    'review_attention_mask': review_encoding['attention_mask'].squeeze(
    'ocr_input_ids': ocr_encoding['input_ids'].squeeze(
    'ocr_attention_mask': ocr_encoding['attention_mask'].squeeze(
    'label': label,
    'product_id': product_id
}

print("✅ Dataset class defined!")

```

✅ Dataset class defined!

```

# =====
# 🧠 CELL 9: ASEEL Model Architecture
# =====

class ASEELModel(nn.Module):
    """
    ASEEL: Multimodal Deep Learning for Counterfeit Product Detection
    Image(1280) + Tabular(32) + Review(768) + OCR(768) = 2848 Tokens
    """

    def __init__(self, num_tabular_features=6, freeze_bert_layers=True):
        super(ASEELModel, self).__init__()

```

```
# 1. IMAGE BRANCH: EfficientNet-B0
efficientnet = models.efficientnet_b0(weights=models.E
if freeze_efficientnet:
    for param in efficientnet.parameters():
        param.requires_grad = False
    for param in efficientnet.features[-2:].parameters
        param.requires_grad = True

self.image_backbone = efficientnet.features
self.image_pool = nn.AdaptiveAvgPool2d(1)
self.image_dim = 1280

# Attention for multiple images
self.image_attention = nn.Sequential(
    nn.Linear(self.image_dim, 128),
    nn.Tanh(),
    nn.Linear(128, 1)
)

# 2. TABULAR BRANCH
self.tabular_branch = nn.Sequential(
    nn.Linear(num_tabular_features, 64),
    nn.BatchNorm1d(64),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(64, 32),
    nn.BatchNorm1d(32),
    nn.ReLU(),
)
self.tabular_dim = 32

# 3. REVIEW BRANCH: BERT
self.review_bert = BertModel.from_pretrained('bert-base
for i, layer in enumerate(self.review_bert.encoder.layer
    if i < freeze_bert_layers:
        for param in layer.parameters():
            param.requires_grad = False
self.review_dim = 768

# 4. OCR BRANCH (shared BERT + projection)
self.ocr_projection = nn.Linear(768, 768)
self.ocr_dim = 768

# 5. FUSION & CLASSIFIER
```

```

fusion_dim = self.image_dim + self.tabular_dim + self.
self.classifier = nn.Sequential(
    nn.Linear(fusion_dim, 512),
    nn.BatchNorm1d(512),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(512, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 1)
)

def forward(self, images, num_valid_images, tabular,
            review_input_ids, review_attention_mask,
            ocr_input_ids, ocr_attention_mask):

    batch_size = images.size(0)
    max_imgs = images.size(1)

    # 1. Image: EfficientNet + Attention Pooling
    imgs_flat = images.view(-1, 3, 224, 224)
    img_features = self.image_backbone(imgs_flat)
    img_features = self.image_pool(img_features).squeeze(-1)
    img_features = img_features.view(batch_size, max_imgs,

    attn_weights = self.image_attention(img_features).squeeze(-1)
    mask = torch.arange(max_imgs, device=images.device).unsqueeze(0)
    attn_weights = attn_weights.masked_fill(~mask, float('-inf'))
    attn_weights = torch.softmax(attn_weights, dim=1).unsqueeze(0)
    img_out = (img_features * attn_weights).sum(dim=1)

    # 2. Tabular
    tab_out = self.tabular_branch(tabular)

    # 3. Review BERT
    review_out = self.review_bert(input_ids=review_input_ids,
                                attention_mask=review_attention_mask)

    # 4. OCR (shared BERT + projection)
    ocr_out = self.ocr_projection(
        self.review_bert(input_ids=ocr_input_ids,
                        attention_mask=ocr_attention_mask)
    )

```

```
# 5. Fusion
fused = torch.cat([img_out, tab_out, review_out, ocr_out])
return self.classifier(fused).squeeze(1)

# Model Summary
model = ASEELModel(num_tabular_features=len(tabular_features))
model = model.to(device)

total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"🧠 ASEEL Model Created!")
print(f"    Total parameters: {total_params:,}")
print(f"    Trainable: {trainable_params:,}")
print(f"    Frozen: {total_params - trainable_params:,}")
print(f"    Fusion: 1280 + 32 + 768 + 768 = 2,848")
del model; torch.cuda.empty_cache()
```

Downloading: "

Warning: You are sending unauthenticated requests to the HF Hub. Please use `huggingface\_hub.utils.\_http:Warning: You are sending unauthenticated requests to the HF Hub. Please use` to suppress this warning.

config.json: 100% 570/570 [00:00<00:00, 79.3kB/s]

model.safetensors: 100% 440M/440M [00:02<00:00, 647MB/s]

Loading weights: 100% 199/199 [00:00<00:00, 1041.51it/s, Materializing param=pooler.dense.weight]

BertModel LOAD REPORT from: bert-base-uncased

Key	Status		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/arc

🧠 ASEEL Model Created!

Total parameters: 115,772,958

Trainable: 42,016,082

Frozen: 73,756,876

Fusion: 1280 + 32 + 768 + 768 = 2,848

```
# =====
# 🏆 CELL 10: Training & Evaluation Functions
```

```
# =====

class EarlyStopping:
    def __init__(self, patience=7, min_delta=0.001):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.early_stop = False
        self.best_model_state = None

    def __call__(self, val_loss, model):
        if self.best_loss is None:
            self.best_loss = val_loss
            self.best_model_state = {k: v.clone() for k, v in model.state_dict().items()}
        elif val_loss > self.best_loss - self.min_delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
                model.load_state_dict(self.best_model_state)
                print(f"🛑 Early stopping! Best loss: {self.best_loss}")
        else:
            self.best_loss = val_loss
            self.best_model_state = {k: v.clone() for k, v in model.state_dict().items()}
            self.counter = 0

def train_one_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    total_loss = 0
    all_preds, all_labels = [], []

    for batch in dataloader:
        images = batch['images'].to(device)
        num_valid = batch['num_valid_images'].to(device)
        tabular = batch['tabular'].to(device)
        r_ids = batch['review_input_ids'].to(device)
        r_mask = batch['review_attention_mask'].to(device)
        o_ids = batch['ocr_input_ids'].to(device)
        o_mask = batch['ocr_attention_mask'].to(device)
        labels = batch['label'].to(device)

        optimizer.zero_grad()
        outputs = model(images, num_valid, tabular, r_ids, r_mask, o_ids, o_mask)
        loss = criterion(outputs, labels)
```



```

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_grad_norm)
        optimizer.step()

    total_loss += loss.item() * len(labels)
    preds = torch.sigmoid(outputs).detach().cpu().numpy()
    all_preds.extend(preds)
    all_labels.extend(labels.cpu().numpy())

avg_loss = total_loss / len(dataloader.dataset)
acc = accuracy_score(all_labels, [1 if p > 0.5 else 0 for p in all_preds])
return avg_loss, acc

def evaluate(model, dataloader, criterion, device):
    model.eval()
    total_loss = 0
    all_preds, all_labels, all_probs = [], [], []

    with torch.no_grad():
        for batch in dataloader:
            images = batch['images'].to(device)
            num_valid = batch['num_valid_images'].to(device)
            tabular = batch['tabular'].to(device)
            r_ids = batch['review_input_ids'].to(device)
            r_mask = batch['review_attention_mask'].to(device)
            o_ids = batch['ocr_input_ids'].to(device)
            o_mask = batch['ocr_attention_mask'].to(device)
            labels = batch['label'].to(device)

            outputs = model(images, num_valid, tabular, r_ids, r_mask, o_ids, o_mask)
            loss = criterion(outputs, labels)

            total_loss += loss.item() * len(labels)
            probs = torch.sigmoid(outputs).cpu().numpy()
            all_probs.extend(probs)
            all_preds.extend([1 if p > 0.5 else 0 for p in probs])
            all_labels.extend(labels.cpu().numpy())

    avg_loss = total_loss / len(dataloader.dataset)
    acc = accuracy_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds)
    auc = roc_auc_score(all_labels, all_probs)
    return avg_loss, acc, f1, auc, all_labels, all_preds, all_probs

```

```
print("✅ Training functions defined!")
```

✅ Training functions defined!

```
# =====
# 🛠️ CELL 11: Transforms & Tokenizer
# =====

train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
print("✅ Transforms & Tokenizer ready!")
```

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 6.63kB/s]

vocab.txt: 232k/? [00:00<00:00, 8.84MB/s]

tokenizer.json: 466k/? [00:00<00:00, 24.8MB/s]

✅ Transforms & Tokenizer ready!

```
# =====
# 🔄 CELL 12: 5-Fold Cross-Validation (FIXED – No Data Leakage)
# =====
# 🛠️ الإصدارات:
# 1. LabelEncoders فقط تُدرَّب على train fold
# 2. StandardScaler فقط يُدرَّب على train fold
# 3. best fold من encoders حفظ
# 4. val معالجة الفئات/البراندات غير المرئية في
```

```

BATCH_SIZE = 16
EPOCHS = 20
LEARNING_RATE = 2e-5
WEIGHT_DECAY = 1e-4
PATIENCE = 7
MAX_IMAGES = 3
N_FOLDS = 5

skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_s
X = cv_data.index.values
y = cv_data['is_authentic'].values

fold_results = []
best_auc = 0
best_model_state = None
best_fold_val_loader = None
best_fold_val_data = None
best_label_encoders = None      # 🛠️ FIX: save from best fold
best_price_scaler = None        # 🛠️ FIX: save from best fold
train_losses = []
val_losses = []

print("=" * 70)
print(f" 🔄 5-FOLD CROSS-VALIDATION (FIXED – No Data Leakage)
print(f" Epochs: {EPOCHS} | Batch: {BATCH_SIZE} | LR: {LEARNIN
print("=" * 70)

for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
    print(f"\n{'-'*50}")
    print(f" 📁 FOLD {fold+1}/{N_FOLDS}")
    print(f"{'-'*50}")

    # نسخ عشان ما نعدّل الأصل
    train_data_cv = cv_data.iloc[train_idx].copy()
    val_data_cv = cv_data.iloc[val_idx].copy()

    # =====
    # 🛠️ FIX #1: Label Encoding – fit on TRAIN only
    # =====
    fold_label_encoders = {}
    for col in categorical_cols:
        le = LabelEncoder()
        le.fit(train_data_cv[col].astype(str))
        fold_label_encoders[col] = le

```

```

# transform train
train_data_cv[f'{col}_encoded'] = le.transform(train_data_cv[col])

# transform val مع معالجة القيم غير المرئية (fallback)
known = set(le.classes_)
val_col = val_data_cv[col].astype(str)
unseen = ~val_col.isin(known)
if unseen.any():
    val_col = val_col.where(~unseen, le.classes_[0])
val_data_cv[f'{col}_encoded'] = le.transform(val_col)

# =====
# 🛠️ FIX #2: Price Scaling – fit on TRAIN only
# =====
price_scaler_fold = StandardScaler()
train_data_cv['price_scaled'] = price_scaler_fold.fit_transform(train_data_cv['price'])
val_data_cv['price_scaled'] = price_scaler_fold.transform(val_data_cv['price'])

# =====
# 🛠️ FIX #3: Class Weights – from TRAIN only
# =====
num_fake = (train_data_cv['is_authentic']==0).sum()
num_real = (train_data_cv['is_authentic']==1).sum()
pos_weight = torch.tensor([num_fake / num_real]).to(device)

print(f" Train: {len(train_data_cv)} ({num_fake} fake, {num_real} real)")
print(f" Val: {len(val_data_cv)} ({val_data_cv['is_authentic'].sum()} real)")
print(f" Price scaler fit on TRAIN: mean={price_scaler_fold.mean_.item():.3f}")
print(f" pos_weight (from TRAIN): {pos_weight.item():.3f}")

train_ds = LuxuryBagDataset(train_data_cv, product_images,
                             tabular_features, train_transform)
val_ds = LuxuryBagDataset(val_data_cv, product_images, token_embeddings,
                           tabular_features, val_transform)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False)

model = ASEELModel(num_tabular_features=len(tabular_features),
                    criterion=nn.BCEWithLogitsLoss(pos_weight=pos_weight) #
                    optimizer=optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY)
                    scheduler=optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=10)
                    early_stopping = EarlyStopping(patience=PATIENCE)

```

```

fold_train_losses = []
fold_val_losses = []

for epoch in range(EPOCHS):
    train_loss, train_acc = train_one_epoch(model, train_loader,
    val_loss, val_acc, val_f1, val_auc, _, _, _ = evaluate
    scheduler.step(val_loss)
    fold_train_losses.append(train_loss)
    fold_val_losses.append(val_loss)

    if (epoch+1) % 5 == 0 or epoch == 0:
        print(f" Epoch {epoch+1:2d} | Train: {train_loss:
        .4f} | Val: {val_loss: .4f} | Acc: {val_acc: .4f} | F1: {val_f1: .4f} | AUC: {val_auc: .4f}")

    early_stopping(val_loss, model)
    if early_stopping.early_stop:
        break

_, val_acc, val_f1, val_auc, labels, preds, probs = evaluate(
fold_results.append({'fold': fold+1, 'accuracy': val_acc,
print(f" 🇮🇹 Fold {fold+1}: Acc={val_acc:.4f} | F1={val_f1:.4f} | AUC={val_auc:.4f}")

# Save best fold model + encoders
if val_auc > best_auc:
    best_auc = val_auc
    best_model_state = {k: v.clone() for k, v in model.state_dict().items()}
    best_fold_val_loader = val_loader
    best_fold_val_data = val_data_cv
    best_label_encoders = fold_label_encoders # 🛠️ F
    best_price_scaler = price_scaler_fold # 🛠️ F
    best_pos_weight = pos_weight
    train_losses = fold_train_losses
    val_losses = fold_val_losses
    print(f" 🏆 New best model! AUC: {val_auc:.4f}")

del model, optimizer, scheduler
gc.collect()
torch.cuda.empty_cache()

# Load best model as final_model
final_model = ASEELModel(num_tabular_features=len(tabular_features))
final_model.load_state_dict(best_model_state)
# NOTE: best_fold_val_loader is the VALIDATION of best fold (used for training)
# The REAL test set (held-out 20%) is evaluated separately in (used for testing)
best_val_loader = best_fold_val_loader
label_encoders = best_label_encoders # for saving/demo

```

```

price_scaler = best_price_scaler
pos_weight = best_pos_weight
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight) # for

# Summary
print("\n" + "=" * 70)
print(" 🇮🇹 5-FOLD CV SUMMARY (FIXED – No Data Leakage)")
print("=" * 70)
results_df = pd.DataFrame(fold_results)
print(f" Accuracy: {results_df['accuracy'].mean():.4f} ± {results_d
print(f" F1 Score: {results_df['f1'].mean():.4f} ± {results_d
print(f" AUC-ROC: {results_df['auc'].mean():.4f} ± {results_d
print(f"\n{results_df.to_string(index=False)}")
print(f"\n🏆 Best Fold AUC: {best_auc:.4f} → loaded as final_m
print(f"🔒 Encoders saved from best fold for deployment")

```

```

=====
🔄 5-FOLD CROSS-VALIDATION (FIXED – No Data Leakage)
Epochs: 20 | Batch: 16 | LR: 2e-05
=====

```

📁 FOLD 1/5

Train: 688 (328 fake, 360 real)
Val: 172 (81 fake, 91 real)
Price scaler fit on TRAIN: mean=1633.15, scale=3271.13
pos_weight (from TRAIN): 0.911

Loading weights: 100% 199/199 [00:00<00:00, 998.34it/s, Materializing param=pooler.dense.we

BertModel LOAD REPORT from: bert-base-uncased

Key	Status		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/ai

Epoch 1	Train: 0.6305/0.5945	Val: 0.5043/0.7558/F1:0.8000/AI
Epoch 5	Train: 0.3124/0.8881	Val: 0.2875/0.8837/F1:0.8913/AI
Epoch 10	Train: 0.2082/0.9375	Val: 0.2345/0.9186/F1:0.9255/AI
Epoch 15	Train: 0.1395/0.9709	Val: 0.1946/0.9302/F1:0.9355/AI
Epoch 20	Train: 0.1081/0.9782	Val: 0.1892/0.9244/F1:0.9312/AI

🇮🇹 Fold 1: Acc=0.9244 | F1=0.9312 | AUC=0.9792

🏆 New best model! AUC: 0.9792

 NEW BEST MODEL: AUC: 0.9792

 FOLD 2/5

Train: 688 (327 fake, 361 real)

Val: 172 (82 fake, 90 real)

Price scaler fit on TRAIN: mean=1646.60, scale=3310.70

pos_weight (from TRAIN): 0.906

Loading weights: 100% 199/199 [00:00<00:00, 927.75it/s, Materializing param=pooler.dense.we

BertModel LOAD REPORT from: bert-base-uncased

Key	Status		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/ai
Epoch 1 | Train: 0.5699/0.6613 | Val: 0.4723/0.8372/F1:0.8511/AU

```
# =====
# 🔄 CELL 12b: Retrain FINAL Model on ALL CV Data
# =====
# ريب موديل نهائي، Fold CV-بعد ما عرفنا الأداء المتوقع من الـ 5
# fold. بدل الاكتفاء بأفضل (80% الـ) cv_data على *كامل* الـ
# (best practice) هذا يستفيد من كل بيانات التدريب → موديل نهائي أقوى
# الـ test (20%) والـ cv_data، تُدرَّب على كامل encoders/scaler الـ

print("=" * 70)
print(" 🔄 RETRAINING FINAL MODEL ON ALL CV DATA (80%)")
print("=" * 70)

# early stopping (لـ) train / 10% val : داخلياً 90 cv_data نقسم
from sklearn.model_selection import train_test_split
final_train, final_val = train_test_split(
    cv_data, test_size=0.10, stratify=cv_data['is_authentic'],
)
final_train = final_train.reset_index(drop=True)
final_val = final_val.reset_index(drop=True)

# 🔧 Encoders فقط final_train تُدرَّب على (no leakage)
final_label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
```

```

le.fit(final_train[col].astype(str))
final_label_encoders[col] = le
final_train[f'{col}_encoded'] = le.transform(final_train[col])
known = set(le.classes_)
vcol = final_val[col].astype(str)
unseen = ~vcol.isin(known)
if unseen.any():
    vcol = vcol.where(~unseen, le.classes_[0])
final_val[f'{col}_encoded'] = le.transform(vcol)

# 🔧 Scaler على final_train فقط
final_scaler = StandardScaler()
final_train['price_scaled'] = final_scaler.fit_transform(final_train['price'])
final_val['price_scaled'] = final_scaler.transform(final_val['price'])

# Class weights من final_train
nf = (final_train['is_authentic']==0).sum()
nr = (final_train['is_authentic']==1).sum()
final_pos_weight = torch.tensor([nf / nr]).to(device)

print(f" Final train: {len(final_train)} | Final val (early-stop) : {len(final_val)}")
print(f" pos_weight: {final_pos_weight.item():.3f}")

# DataLoaders
final_train_ds = LuxuryBagDataset(final_train, product_images,
                                   tabular_features, train_transform)
final_val_ds = LuxuryBagDataset(final_val, product_images, token_embeddings,
                                 tabular_features, val_transform)
final_train_loader = DataLoader(final_train_ds, batch_size=BATCH_SIZE)
final_val_loader = DataLoader(final_val_ds, batch_size=BATCH_SIZE)

# موديل جديد
retrained_model = ASEELModel(num_tabular_features=len(tabular_features))
final_criterion = nn.BCEWithLogitsLoss(pos_weight=final_pos_weight)
final_optimizer = optim.AdamW(filter(lambda p: p.requires_grad,
                                       retrained_model.parameters()),
                                lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY)
final_scheduler = optim.lr_scheduler.ReduceLROnPlateau(final_optimizer, patience=10)
final_early_stop = EarlyStopping(patience=PATIENCE)

final_train_losses = []
final_val_losses = []

for epoch in range(EPOCHS):
    tl, ta = train_one_epoch(retrained_model, final_train_loader, final_val_loader)
    vl, va, vf, vauc, _, _, _ = evaluate(retrained_model, final_val_loader)

```



```

final_scheduler.step(vl)
final_train_losses.append(tl)
final_val_losses.append(vl)
if (epoch+1) % 5 == 0 or epoch == 0:
    print(f" Epoch {epoch+1:2d} | Train: {tl:.4f}/{ta:.4f}")
final_early_stop(vl, retrained_model)
if final_early_stop.early_stop:
    print(f" 🛑 Early stop @ epoch {epoch+1}")
    break

# 🔄 تدريبها (على كل البيانات) ونستبدل الموديل والـ
final_model = retrained_model
label_encoders = final_label_encoders
price_scaler = final_scaler
pos_weight = final_pos_weight
criterion = final_criterion
train_losses = final_train_losses
val_losses = final_val_losses

gc.collect(); torch.cuda.empty_cache()

print("\n ✅ Final model retrained on ALL CV data (80%).")
print(" ✅ This model (not best-fold) will be evaluated on the")
print(" 🇮🇹 5-Fold CV results above show expected performance")

```

🔄 RETRAINING FINAL MODEL ON ALL CV DATA (80%)

Final train: 774 | Final val (early-stop): 86
pos_weight: 0.906

Loading weights: 100% 199/199 [00:00<00:00, 928.97it/s, Materializing param=pooler.dense.weight]

BertModel LOAD REPORT from: bert-base-uncased

Key	Status		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/arc
- | | | |
|----------|----------------------|----------------------------------|
| Epoch 1 | Train: 0.5748/0.6848 | Val: 0.5058/0.7093/F1:0.7525/AUC |
| Epoch 5 | Train: 0.2806/0.8979 | Val: 0.3352/0.8837/F1:0.8913/AUC |
| Epoch 10 | Train: 0.1786/0.9548 | Val: 0.2706/0.8953/F1:0.9032/AUC |

```
Epoch 15 | Train: 0.1251/0.9716 | Val: 0.2776/0.8953/F1:0.9011/AUC
📦 Early stopping! Best loss: 0.2431
📦 Early stop @ epoch 19

✅ Final model retrained on ALL CV data (80%).
✅ This model (not best-fold) will be evaluated on the held-out te
```

```
# =====
# 📦 CELL 13b: Evaluate on HELD-OUT Test Set (ذج ما شافها أبداً)
# =====
# نجوز test set على الـ best fold من encoders + scaler نطبق الـ
# ثم النموذج عليها مرة واحدة فقط - هذي النتائج الحقيقية النزيهة

print("=" * 70)
print(" 📦 EVALUATING ON HELD-OUT TEST SET (never seen during
print("=" * 70)

# المحجوز test set نسخة من الـ
test_eval = test_data_holdout.copy()

# 🔧 فقط - مدربة على best fold (transform encoders نطبق الـ
for col in categorical_cols:
    le = label_encoders[col] # من best fold
    known = set(le.classes_)
    col_vals = test_eval[col].astype(str)
    unseen = ~col_vals.isin(known)
    if unseen.any():
        print(f" ⚠️ {unseen.sum()} test samples have unseen '
        col_vals = col_vals.where(~unseen, le.classes_[0])
        test_eval[f'{col}_encoded'] = le.transform(col_vals)

# 🔧 فقط price scaler من best fold (transform نطبق)
test_eval['price_scaled'] = price_scaler.transform(test_eval[[

# لنبي DataLoader للـ test
test_holdout_ds = LuxuryBagDataset(test_eval, product_images,
                                   tabular_features, val_transf
test_holdout_loader = DataLoader(test_holdout_ds, batch_size=B
                                   shuffle=False, num_workers=2

# المحجوز test set على الـ final_model نقيم
test_loss, test_acc, test_f1, test_auc, y_true, y_pred, y_prob
    final_model, test_holdout_loader, criterion, device)

print(f"\n 🇮🇹 HELD-OUT TEST RESULTS (honest, unbiased):")
```

```

print(f"      Accuracy:  {test_acc:.4f}")
print(f"      F1 Score:   {test_f1:.4f}")
print(f"      AUC-ROC:     {test_auc:.4f}")
print(f"\n{classification_report(y_true, y_pred, target_names=

# نخلي test_loader يشير للـ holdout عشان Cell 14 (per-brand)
test_loader = test_holdout_loader
test_data = test_eval

gc.collect(); torch.cuda.empty_cache()

print("\n  ✅ These are TRUE test results on data the model NE
print("  ✅ No data leakage, no optimistic bias from fold sele

```



EVALUATING ON HELD-OUT TEST SET (never seen during training)



HELD-OUT TEST RESULTS (honest, unbiased):

Accuracy: 0.9395
 F1 Score: 0.9417
 AUC-ROC: 0.9829

	precision	recall	f1-score	support
Counterfeit	0.9238	0.9510	0.9372	102
Authentic	0.9545	0.9292	0.9417	113
accuracy			0.9395	215
macro avg	0.9392	0.9401	0.9395	215
weighted avg	0.9400	0.9395	0.9396	215



These are TRUE test results on data the model NEVER saw during
 No data leakage, no optimistic bias from fold selection.

```

# =====
# 📊 CELL 13: Test Results & Visualizations
# =====

# Results already computed in Cell 13b (held-out test). Reuse

print("=" * 70)
print("  📊 HELD-OUT TEST RESULTS & VISUALIZATIONS")
print("=" * 70)
print(f" Accuracy:  {test_acc:.4f}")
print(f" F1 Score:   {test_f1:.4f}")

```

```

print(f" AUC-ROC: {test_auc:.4f}")
print(f"\n{classification_report(y_true, y_pred, target_names=

# Plots
fig, axes = plt.subplots(2, 2, figsize=(16, 14))

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0,0],
            xticklabels=['Counterfeit', 'Authentic'], yticklabels=['Authentic', 'Counterfeit'])
axes[0,0].set_title('Confusion Matrix', fontsize=14, fontweight='bold')
axes[0,0].set_ylabel('True'); axes[0,0].set_xlabel('Predicted')

# ROC
fpr, tpr, _ = roc_curve(y_true, y_prob)
axes[0,1].plot(fpr, tpr, 'b-', lw=2, label=f'AUC = {test_auc:.4f}')
axes[0,1].plot([0,1],[0,1], 'k--', alpha=0.5)
axes[0,1].set_xlabel('FPR'); axes[0,1].set_ylabel('TPR')
axes[0,1].set_title('ROC Curve', fontsize=14, fontweight='bold')
axes[0,1].legend(); axes[0,1].grid(alpha=0.3)

# PR Curve
prec, rec, _ = precision_recall_curve(y_true, y_prob)
axes[1,0].plot(rec, prec, 'g-', lw=2)
axes[1,0].set_xlabel('Recall'); axes[1,0].set_ylabel('Precision')
axes[1,0].set_title('Precision-Recall Curve', fontsize=14, fontweight='bold')
axes[1,0].grid(alpha=0.3)

# Loss Curves
axes[1,1].plot(range(1, len(train_losses)+1), train_losses, 'b-', lw=2)
axes[1,1].plot(range(1, len(val_losses)+1), val_losses, 'r-', lw=2)
axes[1,1].set_xlabel('Epoch'); axes[1,1].set_ylabel('Loss')
axes[1,1].set_title('Loss Curves (Best Fold)', fontsize=14, fontweight='bold')
axes[1,1].legend(); axes[1,1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(DATA_PATH, 'ASEEL_results.png'), dpi=300)
plt.show()

```



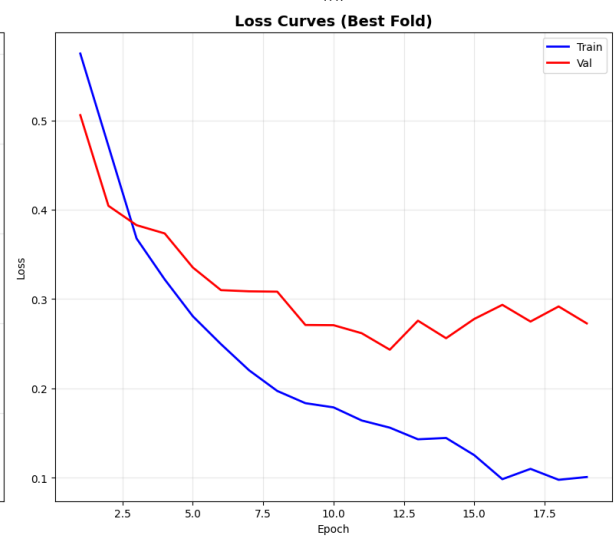
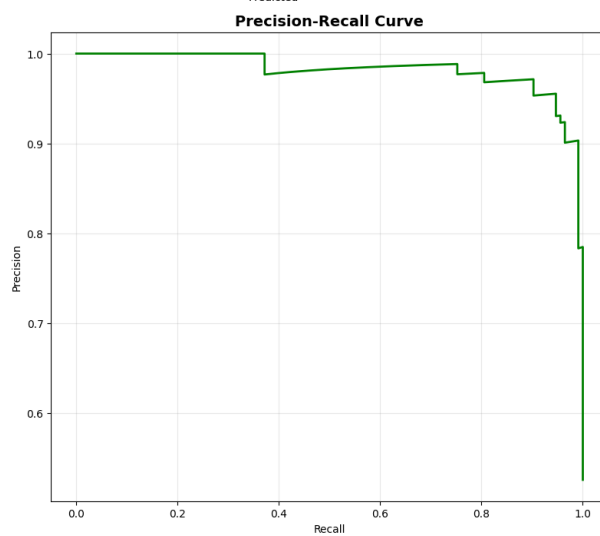
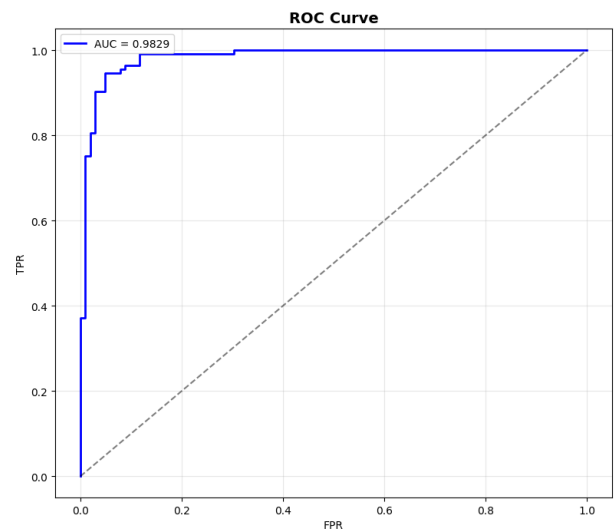
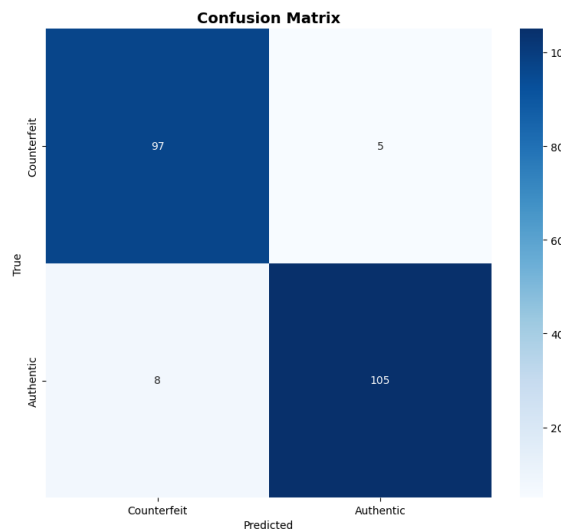
HELD-OUT TEST RESULTS & VISUALIZATIONS

```

Accuracy:  0.9395
F1 Score:  0.9417
AUC-ROC:   0.9829

```

	precision	recall	f1-score	support
Counterfeit	0.92	0.95	0.94	102
Authentic	0.95	0.93	0.94	113
accuracy			0.94	215
macro avg	0.94	0.94	0.94	215
weighted avg	0.94	0.94	0.94	215



=====

```

#  CELL 14: Per-Brand Results
# =====

final_model.eval()
brand_results = []

with torch.no_grad():
    for batch in test_loader:
        images = batch['images'].to(device)
        num_valid = batch['num_valid_images'].to(device)
        tabular = batch['tabular'].to(device)
        r_ids = batch['review_input_ids'].to(device)
        r_mask = batch['review_attention_mask'].to(device)
        o_ids = batch['ocr_input_ids'].to(device)
        o_mask = batch['ocr_attention_mask'].to(device)
        labels = batch['label']
        pids = batch['product_id']

        outputs = final_model(images, num_valid, tabular, r_ids, r_mask, o_ids, o_mask)
        probs = torch.sigmoid(outputs).cpu().numpy()

        for pid, lbl, prob in zip(pids, labels.numpy(), probs):
            brand_results.append({
                'Id': pid.item() if torch.is_tensor(pid) else pid,
                'true': lbl, 'pred': 1 if prob > 0.5 else 0, 'prob': prob
            })

brand_df = pd.DataFrame(brand_results).merge(data[['Id', 'brand']])

print("=" * 70)
print("  PER-BRAND RESULTS")
print("=" * 70)

brand_metrics = []
for brand in sorted(brand_df['brand'].unique()):
    b = brand_df[brand_df['brand']==brand]
    if len(b) < 2: continue
    acc = accuracy_score(b['true'], b['pred'])
    f1_b = f1_score(b['true'], b['pred'], zero_division=0)
    try: auc_b = roc_auc_score(b['true'], b['prob'])
    except: auc_b = 0
    brand_metrics.append({'Brand': brand, 'N': len(b), 'Accuracy': acc, 'F1': f1_b, 'AUC': auc_b})
    print(f" {brand:15s} | N={len(b):3d} | Acc={acc:.4f} | F1={f1_b:.4f} | AUC={auc_b:.4f}")

bm_df = pd.DataFrame(brand_metrics)

```

```

fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(bm_df))
w = 0.25
ax.bar(x-w, bm_df['Accuracy'], w, label='Accuracy', color='#3498db')
ax.bar(x, bm_df['F1'], w, label='F1', color='#2ecc71')
ax.bar(x+w, bm_df['AUC'], w, label='AUC', color='#e74c3c')
ax.set_xticks(x); ax.set_xticklabels(bm_df['Brand'], rotation=45)
ax.set_ylabel('Score'); ax.set_title('Per-Brand Performance', fontweight='bold')
ax.legend(); ax.set_ylim(0,1.1); ax.grid(alpha=0.3, axis='y')
plt.tight_layout()
plt.savefig(os.path.join(DATA_PATH, 'ASEEL_per_brand.png'), dpi=300)
plt.show()

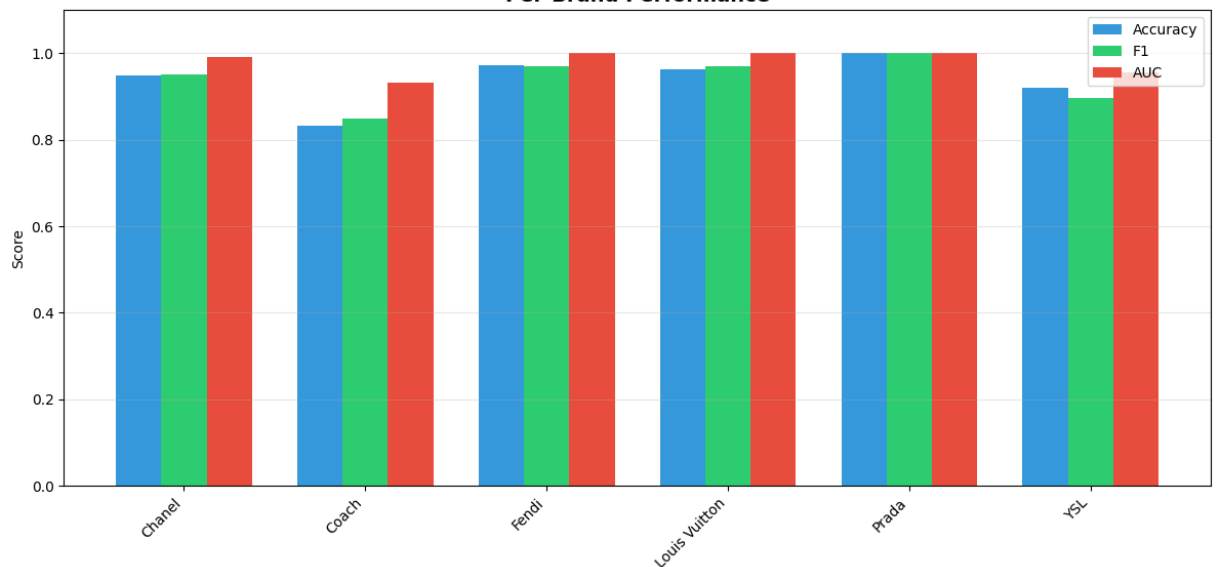
```



PER-BRAND RESULTS

Chanel	N= 39	Acc=0.9487	F1=0.9524	AUC=0.9921
Coach	N= 36	Acc=0.8333	F1=0.8500	AUC=0.9333
Fendi	N= 37	Acc=0.9730	F1=0.9697	AUC=1.0000
Louis Vuitton	N= 27	Acc=0.9630	F1=0.9714	AUC=1.0000
Prada	N= 38	Acc=1.0000	F1=1.0000	AUC=1.0000
YSL	N= 38	Acc=0.9211	F1=0.8966	AUC=0.9554

Per-Brand Performance



```

# =====
# 🔍 CELL 16: Grad-CAM Visualization (FIXED – based on real pr
# =====
# 🔧 قياسي للنموذج PREDICTION يعتمد على الـ Grad-CAM الإصحاح: الـ
# image وليس متوسط الـ (authentic/counterfeit) المخرج النهائي)

class GradCAM:
    """Grad-CAM (الحكم على الأصالة) المخرج النهائي للنموذج .
    def __init__(self, model, target_layer):
        self.model = model
        self.gradients = None
        self.activations = None
        self.h1 = target_layer.register_forward_hook(self._save_
        self.h2 = target_layer.register_full_backward_hook(self

    def _save_activation(self, module, inp, out):
        self.activations = out.detach()

    def _save_gradient(self, module, grad_in, grad_out):
        self.gradients = grad_out[0].detach()

    def generate(self, single_image, full_inputs):
        """
        single_image: [224 ,224 ,3 ,1] الصورة المراد تفسيرها
        full_inputs: dict فيه كل مدخلات النموذج للمنتج (tabular,
        """
        self.model.eval()
        self.model.zero_grad()

        # prediction نمرر المنتج كامل عبر النموذج للحصول على الـ
        # (num_valid=1) نستخدم الصورة الواحدة كأنها المنتج
        img_input = single_image.unsqueeze(1) # [1, 1, 3, 224
        num_valid = torch.tensor([1], device=single_image.device)

        logit = self.model(
            img_input, num_valid,
            full_inputs['tabular'],
            full_inputs['review_input_ids'], full_inputs['review

```



```

        full_inputs['ocr_input_ids'], full_inputs['ocr_attn']
    )
    # ✂ من المخرج النهائي (الحكم) - مش متوسط gradient الى
    score = logit.squeeze()
    score.backward(retain_graph=True)

    # weights = لكل قناة gradient متوسط
    weights = self.gradients.mean(dim=[2, 3], keepdim=True)
    cam = torch.relu((weights * self.activations).sum(dim=[2, 3]))
    cam = cam - cam.min()
    if cam.max() > 0:
        cam = cam / cam.max()
    return cam, torch.sigmoid(score).item()

def remove_hooks(self):
    self.h1.remove(); self.h2.remove()

grad_cam = GradCAM(final_model, final_model.image_backbone[-1])

# HELD-OUT test set نستخدم عينات من الى
sample = test_eval.head(8)

fig, axes = plt.subplots(2, 4, figsize=(20, 10))

for idx, (_, row) in enumerate(sample.iterrows()):
    if idx >= 8: break
    img_paths = product_images.get(row['Id'], [])
    if not img_paths:
        continue

    orig_img = Image.open(img_paths[0]).convert('RGB')
    img_tensor = val_transform(orig_img).unsqueeze(0).to(device)

    # (tabular + text + ocr) نجهز باقي مدخلات المنتج
    tab = torch.tensor(row[tabular_features].values.astype(np.float32))
    review_enc = tokenizer(str(row['combined_reviews']), max_length=128,
                           padding='max_length', truncation=True)
    ocr_enc = tokenizer(str(row['ocr_text']), max_length=128,
                        padding='max_length', truncation=True,
                        return_tensors='pt')
    full_inputs = {
        'tabular': tab,
        'review_input_ids': review_enc['input_ids'].to(device),
        'review_attention_mask': review_enc['attention_mask'].to(device),
        'ocr_input_ids': ocr_enc['input_ids'].to(device),
        'ocr_attention_mask': ocr_enc['attention_mask'].to(device)
    }

```

```

}

cam, pred_prob = grad_cam.generate(img_tensor, full_inputs)
cam_np = cam.squeeze().cpu().numpy()
if cam_np.ndim > 2:
    cam_np = cam_np.reshape(cam_np.shape[-2], cam_np.shape[-1])
cam_resized = np.array(Image.fromarray((cam_np*255).astype(np.uint8)))

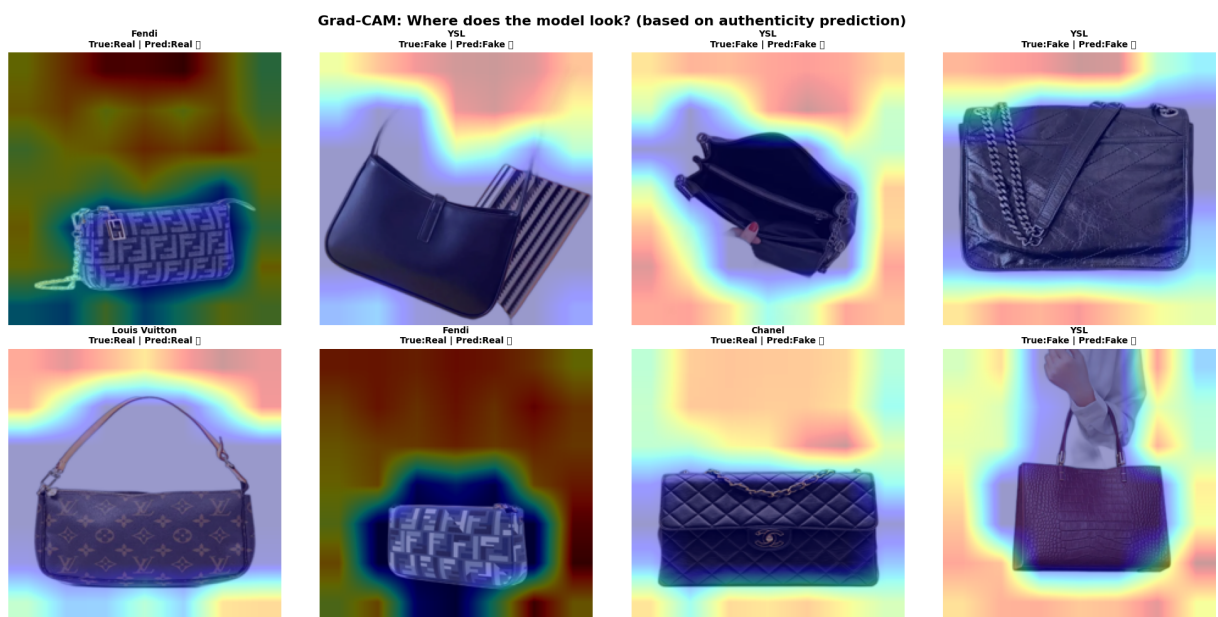
ax = axes[idx//4][idx%4]
ax.imshow(orig_img.resize((224,224)))
ax.imshow(cam_resized, alpha=0.4, cmap='jet')
true_label = "Real" if row['is_authentic']==1 else "Fake"
pred_label = "Real" if pred_prob >= 0.5 else "Fake"
correct = "✅" if (pred_prob>=0.5)==(row['is_authentic']==1) else "❌"
ax.set_title(f"{row['brand']}\nTrue:{true_label} | Pred:{pred_label} | Correct:{correct}"
             , fontsize=10, fontweight='bold')
ax.axis('off')

plt.suptitle('Grad-CAM: Where does the model look? (based on authenticity prediction)'
             , fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig(os.path.join(DATA_PATH, 'ASEEL_gradcam.png'), dpi=300)
plt.show()

grad_cam.remove_hooks()

print("🔍 Grad-CAM Analysis (based on real predictions):")
print("✅ Heatmap shows regions influencing the authenticity prediction")
print("✅ Focus on logo/stitching/hardware → model learns to recognize authentic details")

```



🔍 Grad-CAM Analysis (based on real predictions):

Grad-CAM Analysis (based on test predictions):

- ✓ Heatmap shows regions influencing the authenticity decision
- ✓ Focus on logo/stitching/hardware → model learns real features

```
# =====
# 📁 CELL 17: Save Model & Artifacts
# =====

SAVE_PATH = os.path.join(DATA_PATH, 'ASEEL_model')
os.makedirs(SAVE_PATH, exist_ok=True)

# 1. Model Weights
torch.save({
    'model_state_dict': final_model.state_dict(),
    'num_tabular_features': len(tabular_features),
    'tabular_features': tabular_features,
    'epochs_trained': len(train_losses),
    'test_accuracy': test_acc,
    'test_f1': test_f1,
    'test_auc': test_auc,
}, os.path.join(SAVE_PATH, 'aseel_model.pth'))

# 2. Encoders & Scaler
```

```

with open(os.path.join(SAVE_PATH, 'label_encoders.pkl'), 'wb')
    pickle.dump({'label_encoders': label_encoders, 'price_scaler': price_scaler,
                'tabular_features': tabular_features}, f)

# 3. Training Results
with open(os.path.join(SAVE_PATH, 'training_results.pkl'), 'wb')
    pickle.dump({
        'cv_results': fold_results,
        'test_results': {'accuracy': test_acc, 'f1': test_f1,
                        'y_true': y_true, 'y_pred': y_pred, 'y_score': y_score},
        'train_losses': train_losses, 'val_losses': val_losses,
        'brand_metrics': brand_metrics,
    }, f)

# 4. Full Model
torch.save(final_model, os.path.join(SAVE_PATH, 'aseel_full_model.pth'))

print("=" * 70)
print(f" 📁 ALL SAVED TO: {SAVE_PATH}")
print("=" * 70)
print(" 📁 aseel_model.pth           → weights + metadata")
print(" 📁 aseel_full_model.pth       → complete model")
print(" 📁 label_encoders.pkl         → encoders & scaler")
print(" 📁 training_results.pkl       → all metrics")

```

```

=====
📁 ALL SAVED TO: /content/drive/MyDrive/ASEEL_model
=====

```

```

📁 aseel_model.pth           → weights + metadata
📁 aseel_full_model.pth       → complete model
📁 label_encoders.pkl         → encoders & scaler
📁 training_results.pkl       → all metrics

```

```
# =====  
# 🔄 CELL 18: Load Model for Inference (Reference)  
# =====  
  
def load_aseel_model(model_path, encoders_path, device='cuda')  
    with open(encoders_path, 'rb') as f:  
        artifacts = pickle.load(f)  
  
        checkpoint = torch.load(model_path, map_location=device)  
        model = ASEELModel(num_tabular_features=checkpoint['num_tal  
        model.load_state_dict(checkpoint['model_state_dict'])  
        model.to(device).eval()  
  
        print(f"✅ Loaded! Acc={checkpoint['test_accuracy']:.4f},  
        return model, artifacts  
  
# Usage:  
# model, artifacts = load_aseel_model('ASEEL_model/aseel_model  
print("✅ Inference code ready!")
```

✅ Inference code ready!